Koord: a language for programming and verifying distributed robotics applications

Anonymous Author(s)

Abstract

A program for a robot in an ensemble interacts with its physical environment using sensors and actuators, and with other robots, over a network. Current languages and tools do not provide the necessary platform-independent abstractions nor the portability. Therefore, developing these applications require detailed knowledge of control theory, path planning, network protocols, and numerous platform-specific details. We present a domain specific language called *Koord* which abstracts platform-specific functions for sensing, communication, and low-level control, and makes the platform-independent control and coordination code portable and modularly verifiable. *Koord* raises the level of abstraction in programming by providing distributed shared memory for coordination and port interfaces for sensing and control. We present the complete formal executable semantics of *Koord* in the $\mathbb{K}$ framework. With the symbolic execution provided by the $\mathbb{K}$ engine, we can identify all proof obligations for formally verifying *Koord* applications. Platform independent proof obligations can be discharged using Z3 and platform specific proof obligations can be tested by deploying the applications on the *Koord* simulator and on actual vehicle and quadcopter platforms. We illustrate the power of *Koord* by creating and verifying three applications: formation flight, distributed task allocation, and distributed mapping.

Keywords robotics, distributed, verification, ROS

1 Introduction

Motivation and goals Distributed robotic applications could transform manufacturing [20, 33], transportation [21, 23], agriculture [7, 36], delivery [27], and mapping [39]. Following the trends in cloud, mobile, and machine learning applications, programmability is key in unlocking this potential as robotics platforms become more open and hardware developers shift to the applications marketplace. Available domain specific languages (DSL) for robotics are tightly coupled with platforms, and they combine low-level sensing, communication, and control tasks with the application-level logic (see Section 6 for more details). This tight-coupling and the attendant lack of abstraction hinders application development on all fronts—portability, code reuse, and verification and validation (V&V).

In particular, formal reasoning about a collection of robots communicating, coordinating, and interacting with a physical environment is complexified by cyber-physical interactions. Correctness under concurrency and asynchrony are prominent research problems in distributed computing. Correctness under noise, disturbances, and imprecise platform (plant) models is studied intensively by roboticists and control theorists. The analysis techniques from these communities are based on very different formal models and mathematics, and both would be necessary to provide satisfactory safety guarantees for distributed robotic applications. Our vision is *not* to combine all of the above in an *all encompassing formalism*, but to create a language that separates the concerns to divide and conquer using existing analyses from both communities.

Our goal is to design abstractions and language constructs that separate the *platform-independent* decision and coordination from *platform-dependent* actions. For example, in an application for distributed package delivery with mobile robots, the code for waypoint assignment, load-balancing, and handling failures should abstract away implementations of way-point tracking controllers and functions for navigation and communication. Such application code will be portable, with appropriate platform-specific implementations of these abstractions, e.g., steering controller for car, thrust controller for a quadcopter, and GPS or indoor localization subroutines. In addition, this separation will enable the joining of analysis techniques. For example, in this paper we show how symbolic executions for the application program can be combined with model-free and data-driven analysis of the platform-specific controllers.

Approach In this paper, we present *Koord*: a language and supporting verification and testing tools for programming distributed robotic systems. *Koord* language is designed with following three kinds of users in mind.

- *For the application developer*, our *Koord* language provides key abstractions (*sensor and actuator ports, distributed shared*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
, ,
© Copyright held by the owner/author(s). Publication rights licensed to ACM.

memory, and synchronous execution) to develop robot applications that interact with the the physical environment, and other participating robot programs.

- *For the V&V engineer*, the $\mathbb{K}$-based formal executable semantics we have developed for *Koord* and our Z3-based prover, can help discharge key invariants of the *Koord* applications. This verification process also helps identify the *platform-dependent* proof obligations that have to be discharged or validated through simulation and testing.
- *For the platform engineers* deploying the robot applications, our abstraction makes the *Koord* programs portable across platforms. Our high-fidelity *Koord* simulator can be used in conjunction with other reachability analysis tools to test and validate the platform-dependent proof obligations.

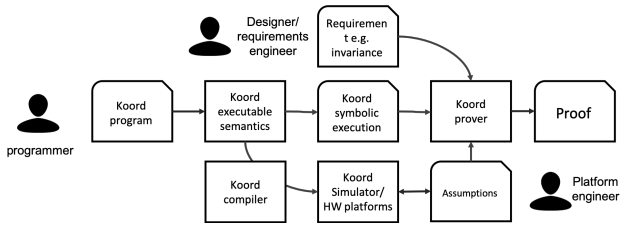

Figure 1. *Koord* language enables programmers to develop distributed robotics applications using a high-level language and prove properties using symbolic executable semantics in $\mathbb{K}$. Platform specific assumptions are abstracted and can be checked using simulations and hardware deployments.

Contributions and design decisions We have baked-in several abstractions in the design of *Koord* to simplify the distributed robotics programming. Our design of *Koord* creates a separation of platform-specific from platform-independent concerns. The formal semantics of *Koord* enables our formal analysis to benefit from this separation of concerns.

- We introduce an abstract interface of *sensor* and *actuator ports* through which a *Koord* program interacts with its environment. The program can read from sensor ports to receive updates about its environment; and it can write to actuator ports to direct the low-level controllers and actuators. Beyond the names and types of these ports, the abstract interface may specify additional *port assumptions* that these ports should satisfy. Application developers get to use these assumptions to reason about the *Koord* application, while platform engineers ensure that these assumptions are met when implementing the interface for each platform. These interfaces allow deploying and simulating the same *Koord* program on heterogeneous devices without any alteration, and thus shorten the test-debug-deployment cycle.
- We provide a *distributed shared memory (DSM)* construct for *Koord* applications on different robots to communicate with each other. This makes *Koord* applications very succinct (see examples in Figure 3), and raises the level of abstraction for the programmers beyond sockets, message queues, ROS topics [35], etc. We have developed an implementation of distributed shared memory using UDP messages.
- We develop the executable $\mathbb{K}$ semantics [37] of *Koord*. To our knowledge this is the first formalization of a programming language for distributed cyber-physical systems which has also been deployed on actual platforms. Our executable semantics of *Koord* in $\mathbb{K}$ assumes a *synchronous round-by-round computation model for the distributed system.* Each round lasts for a fixed time period, and all robots synchronize at the beginning of each round. This synchronous model is a restrictive but standard model for distributed systems [3, 26]. While it does not completely eliminate concurrency control, it significantly simplifies programming and verification. *Koord* provides constructs for mutual exclusion as additional mechanisms for concurrency control with shared memory mentioned above. This model can be implemented under typical synchrony assumptions for multi-robot networks.[1]
- We present a $\mathbb{K}$ symbolic execution-based formal verification methodology which does *not* require explicit dynamical models of the platform. In practice, a detailed model of the platform (e.g., dynamic models for cars, wheel friction, engine torque, etc.) may not be available. The *Koord* semantics is parameterized so that any available model or an actual black-box executable for the platform can be plugged in, and our formal analyses facilitate both inductive invariant checking and state space exploration with black-box dynamics.
- We identify and separate platform-independent from platform-dependent *proof obligations* derived from our formal analysis of *Koord* semantics. We propose an approach to discharge the former obligations with existing SMT solvers automatically. On the one hand, platform dependent proof obligations can suggest the infeasibility of implementing such systems, when they are difficult to discharge or quickly violated in simulation or testing. On the other hand, these proof obligations can serve as contracts for sensors, drivers, and operating system modules that are outside the purview of application developers. An intended useful outcome of this methodology is a list of formal assumptions for the platform or implementation-specific components.

These choices do leave out issues related to robot failure and dynamic joins and leaves from the current semantics, which could be included in the future. Even without tackling these, *Koord* tools can analyze interesting deployed applications such as distributed formation (a set of robots coordinating to form shapes), task allocation (robots performing tasks mutually exclusively and without collision), and mapping (robots collaboratively mapping a given space) in Section 5.

2 Overview and an example

We will discuss the key features of the *Koord* language and programming system with an example. The *Koord* application LineForm shown in Figure 3 implements a simple formation control protocol of the type used for drone shows like the one seen in Figure 2. LineForm makes an arbitrary

[1]Bounded message delays and clocks with bounded drifts.

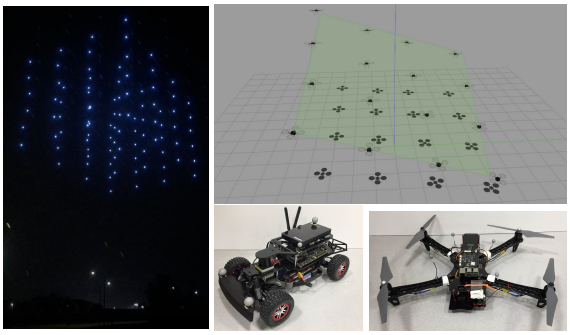

Figure 2. Swarm formation show by FireFly Inc. (*Left*). Simulation of shape formation (*Top Right*). Cars and quadcopters we can deploy and test *Koord* programs.

number of robots (drones) line up uniformly between two extremal robots. Small modifications to the code make the drones form other shapes like squares, cubes, and stars. The *Koord* programming tools can compile and deploy this code on a heterogeneous fleet of robots platforms; they can help automate the verification of these types of applications by decomposing the proofs into platform dependent and independent proof obligations. Finally, the *Koord* simulator can help find violations of assumptions made for verification.

2.1 *Koord* language

Koord is a high-level, event-driven language in which application programs use *shared variables* for coordination across robots and *ports* for interaction with hardware-specific subroutines. In a distributed robotics setting, instances of the same *Koord* program are executed by each participating robot to solve problems collectively.

```
1  using Motion:
2    sensors: Point position
3    actuators: Point target
4
5  allread: Point x[N_SYS]
6  init:
7    x[pid] = Motion.position
9  TargetUpdate:
10   pre: !(pid == N_SYS − 1 or pid == 0)
11   eff:
12     Motion.target = (x[pid+1] + x[pid−1]) / 2
13     x[pid] = Motion.position
```

Figure 3. *Koord* program LineForm for a set of robots to form a line.

Modules and port abstractions. A *Koord* program interacts with the sensors and low-level controllers of the robot platform through *sensor* and *actuator* ports. The program can read data from the sensor ports and can write data to actuator ports. For example, LineForm uses a *module* (library) called *Motion* which provides a sensor port called *position* that publishes the robot's position, and an actuator port called *target* for specifying a target position. Thus, these ports provide an abstraction over various possible sensor and controller implementations and environments. Implementations of the modules are part of the *Koord runtime system* and they implement hardware specific functions. For example, our implementation of the *Motion* module for a quadcopter uses an indoor camera based positioning system to update the *position* port, and it uses an RRT based [25] path planner and motion controller. The *Motion* module abstraction is implemented for a small racing vehicle platform using the same indoor positioning system but a different model-predictive controller.

Local and shared variables. *Koord* programs can have *local* variables similar to most programming languages. In addition, they can also use *shared* variables for participating robots to communicate with each other. At Line 5 in Figure 3, an **allread** variable, x, is a shared array which all robots can read from, but each robot `pid` can only write to x[`pid`]. This shared array is used to share the current position of each robot with all other robots. LineForm uses (a) the unique integer identifier `pid` for itself and (b) the number `N_SYS` of all participating robots. A detailed list of system parameters will be discussed later in Section 3. For multiple robot programs writing to shared variables *Koord* provides concurrency control with mutual exclusion and **atomic** blocks, (see Task and Mapping applications in Section 4).

2.2 Semantics and invariant properties

We have developed the full semantics of *Koord* using $\mathbb{K}$ [37], and present the details in Section 3. The execution semantics of any applications for multi-robot systems are complicated by issues of asynchrony, consistency of shared memory, and interactions between software and the physical environment. The $\mathbb{K}$ rewriting engine makes the formal language semantics *executable*, and enables exhaustive exploration of nondeterministic behaviors of *Koord* applications. In Section 4, we present a method for checking invariant properties using symbolic execution of the *Koord* semantics in $\mathbb{K}$.

For LineForm, a natural requirement is to restrict all robots to stay within a certain safe area, at all times (Geofencing). More precisely, given a (hyper)rectangle $rect(x_{min}, x_{max})$ defined by its two corners x_{min} and x_{max}, if all robots are initialized within the rectangle, then all robots should always stay in the rectangle. This requirement can be stated as:

Invariant 1. $$\bigwedge_{i \in \mathrm{ID}} \left(\begin{array}{l} M.pos_i \in rect(x_{min}, x_{max}) \\ \wedge\, x[i] \in rect(x_{min}, x_{max}) \end{array} \right)$$

where $M.pos$ is the shorthand for $Motion.position$.

Using *Koord*'s supporting proof tools, an invariant like the above can be established in two steps: first, assuming that all the robot positions are in $rect(x_{min}, x_{max})$, we show that the targets computed by LineForm are also in $rect(x_{min}, x_{max})$. The $\mathbb{K}$ semantics of *Koord* allows us to construct the symbolic post states of the *TargetUpdate* event and prove Invariant 1 using the *Koord* prover (Figure 4) as detailed in Section 4.

The second step is to show that for any robot, assuming that the computed targets are in $rect(x_{min}, x_{max})$, the controller implementing *Motion* module indeed keeps the robot inside $rect(x_{min}, x_{max})$. For this step, one has to reason about how each robot moves when its implementation of the *Motion* module is given a *target*. *Koord* helps identify and decompose the overall proof into assumptions that the

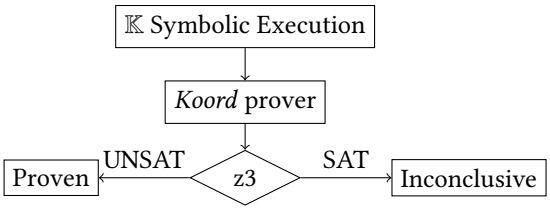

Figure 4. 𝕂 semantics based invariant checking for *Koord*.

Module implementations need to guarantee. For example, we can state the key assumption needed for Invariant 1 as:

Port Assumption 1.

$$\forall t \in [0, \delta], f(M.pos, M.tgt, t) \subseteq rect(M.pos, M.tgt),$$

where $M.tgt$ is the shorthand for $Motion.target$, f is a function giving the position of the robot at time t, moving to $M.tgt$, from $M.pos$. This assumption states that the robot's *Motion* module should ensure that it is moving within the bounding rectangle between its position and target within the duration of a round. In Section 4, we will discuss how these types of assumptions about the control system can be discharged using verification engines for reasoning about continuous behavior of dynamical systems.

2.3 Simulation based assumption validation

Assumption 1 may appear benign at a glance, but it may be violated in some conditions. Using the high-fidelity *Koord* simulator, a designer can gain insights about when such assumptions are violated. The simulator executes complied *Koord* code together with detailed physical models for the robots, ROS-based interactions with sensor models, and UDP-based message passing. A Gazebo visualization of the simulator's output for 16 robots executing a 3D version of LineForm is shown in Figure 2. Users can also use the simulator to detect early in the development process if the assumptions for correctness are too strong under specific scenarios, and revise the assumptions iteratively. Using the *Koord* simulator in conjunction with other verification tools, we can discover that Assumption 1 is violated in three rather common scenarios: First, if a robot has to avoid obstacles, then it may have to go around the obstacle and hence out of the bound. Second, the assumption fails for robots with nonholonomic dynamics such as wheeled robots. Third, the inertia of the robot may force it go out of bound temporarily. We see through an example in Section 4 how the third scenario can be detected.

2.4 Compilation and deployment.

In addition to the formal language, semantics, analysis, and simulation, our complete tool chain also includes compilation and deployment to heterogeneous platforms including drones and race cars. Once developers install our ROS [35] based run-time libraries (middleware) on a platform and provide a device specific configuration denoting the mapping from *Koord* module ports to low level sensor and actuator ROS messages, our port based abstraction (module) then allows the same *Koord* program to run on this platform. Detailed description of the tool chain is available in [4].[2]

3 *Koord* language design

In this section, we present the syntax and the semantics of *Koord*. When a *Koord* application is deployed on a fleet of `N_SYS` robots, each robot runs an instance of the same program. There is a known set of identifiers $\texttt{ID} = \{0, 1, \ldots, \texttt{N_SYS}-1\}$, and each robot is assigned a unique index $\texttt{pid} \in \texttt{ID}$. The execution of the *Koord* program advances in a synchronous, *round-by-round* fashion, where each round lasts for δ time, and $\delta > 0$ is a platform specific execution parameter. During this period, the robots compute, move, and communicate with each other through distributed shared memory.

3.1 Syntax

Figure 5 shows the partial grammar of *Koord* syntax. Each robot program consists of (a) declarations of the interfaces between the program and the sensor/actuator modules, (b) declarations of shared and local program variables, and (c) events, consisting of preconditions and effects. Robot programs (rule *Program*) first can import sensor/actuator modules. The module import grammar production specifies the interfaces or ports: it contains all input and output ports for actuators (*APorts*) and sensors (*SPorts*) that the program uses. To summarize, there are following three types of names for reading/writing values:

(i) *Sensor and actuator ports* are used to read from sensor ports and write to actuator ports of controllers.
(ii) *Local program variables* record the state of the program.
(iii) *Distributed shared variables* are used for coordination across robots. All shared variables can be read by all participating robots; an **allwrite** variables can be written by any participating robot; while an **allread** variables can be written only by a single-writer.

User can then optionally specify a statement to set the initial values of program variables (rule *Init*). The main body of the program is a sequence of events (rule *Event*) which include a Boolean **pre**condition and an **eff**ect. The effect of an event is also a statement (rule *Effect*). We skip the syntax rules for statements, expressions, data types, and functions due to the page limit. A statement (rule *Stmt*) in *Koord* resembles those in most imperative languages and includes conditional statements, function calls, assignments, blocks of statements, atomic statements for mutual exclusion, etc. Mutual exclusion is always an essential feature when shared variables are involved. *Koord* provides a locking mechanism using the keyword **atomic** to update the shared variable safely. The user can also define functions and abstract data types (tuples of the basic data types).

In the syntax presented in Figure 5, given an nonterminal *NT*, $NT^?$ means that it is optional in the syntax at that

[2]Under submission, anonymized for the purposes of double blind reviewing.

Program	::=	*Def* Import* DeclBlock Init? Event⁺*
Def	::=	*TypeDef* | *FuncDef*
Import	::=	**using** *identifier* **:** *SPorts APorts*
SPorts	::=	**sensors :** *VarDecl⁺*
APorts	::=	**actuators :** *VarDecl⁺*
DeclBlock	::=	*AWDecl? ARDecl? LocalDecl?*
AWDecl	::=	**allwrite :** *VarDecl⁺*
ARDecl	::=	**allread :** *ARVarDecl⁺*
LocalDecl	::=	**local :** *VarDecl⁺*
VarDecl	::=	*Type identifier* | *Type identifier = Val*
ARVarDecl	::=	*Type identifier* **[N_SYS]**
Init	::=	**init :** *Stmt*
Event	::=	*identifier* **:** *Precond Effect*
Precond	::=	**pre :** *Expr*
Effect	::=	**eff :** *Stmt*
Stmt	::=	**atomic** *Stmt* | *Stmt StmtList* | ...

Figure 5. Partial *Koord* syntax rules.

position, NT^* refers to zero or more occurrences, and NT^+ refers to one or more occurrences. The expression $(E1 \mid E2)$ denotes that one can use either $E1$ or $E2$.

3.2 Configurations

The semantics of a *Koord* program execution is based on synchronous rounds divided into *event transitions* and *environment transitions* that update the *system configuration*. In each round, each robot performs at most one event. The update performed by a single robot executing an event is modeled as an instantaneous transition that updates the program variables; however, different events executed by the different robots may interleave in an arbitrary sequence. In between the events of successive rounds, $\delta > 0$ duration of time elapses, the program variables remain constant while the values held by the sensor and actuator ports may change. These are modeled as environment transitions that advance time as well as the sensor and actuator ports. Thus, each round consists of a burst of (at most N_SYS) event transitions followed by an environment transition. This is a standard model for synchronous distributed systems where the speed of computation is much faster than the speed of communication and physical movement [3, 26].

We now describe the system state, or *system configurations* which we use to formalize *Koord* semantics.

System configurations. A *system configuration* is a tuple $\boldsymbol{c} = (\{L_i\}_{i \in \mathtt{ID}}, S, \tau, turn)$, where

(i) $\{L_i\}_{i \in \mathtt{ID}}$ or $\{L_i\}$ in short is an indexed set of *robot configurations*–one for each participating robot. L_i refers to the configuration of the i-th element, i.e., the i-th robot in the system.
(ii) $S : Var \mapsto Val$ is the *global context*, mapping all shared variable names to their values.
(iii) $\tau \in \mathbb{R}_{\geq 0}$ is the *global time*.

$$\frac{\langle S, L, St\rangle \rightarrow_{stmt} \langle S', L', St'\rangle}{\langle S, L, St\ StList\rangle \rightarrow_{stmt} \langle S', L', St'\ StList\rangle}\ \textsc{StmtSeq1}$$

$$\langle S, L, \cdot\ StList\rangle \rightarrow_{stmt} \langle S, L, StList\rangle\ \textsc{StmtSeq2}$$

$$\frac{x \in Keys(S) \wedge x \in Keys(L_i.M) \wedge L'_i.M = L_i.M[x \mapsto v]}{\langle S, L_i, x = v\rangle \rightarrow_S \langle S[x \mapsto v], L'_i, \cdot\rangle}\ \textsc{SvarAssign}$$

$$\frac{x \notin Keys(S) \wedge x \in Keys(L_i.M) \wedge\ L'_i.M = L_i.M[x \mapsto v]}{\langle S, L_i, x = v\rangle \rightarrow_S \langle S, L'_i, \cdot\rangle}\ \textsc{LvarAssign}$$

Figure 6. Example statement level semantic rules for *Koord*.

(iv) $turn \in \{\mathtt{prog}, \mathtt{env}\}$ is a binary *bookkeeping* variable determining whether program or environment transitions are being processed.

Bookkeeping variables are invisible in the language syntax, and only used in the semantics.

Robot configurations. A *robot configuration* is used to specify the semantics of each robot. As a *Koord* program is run on a system of robots, each participating robot would have its own set of module ports and local variables, along with a local copy of each shared variable. Given a *Koord* program P, we can define *Var* be the set of variables, *Val* be the set of values that an expression in *Koord* can evaluate to, *CPorts* be the set of sensor and actuator ports of the controller being used, and *Events* the set of events in P. A robot configuration is a tuple $L = (M, cp, turn)$, where

(i) $M : Var \mapsto Val$ is its *local context* mapping both local and shared variables to values. Note that this implies M includes a copy of shared variable values.
(ii) $cp : CPorts \mapsto Val$ is the mapping of sensor and actuator ports to values.
(iii) $turn \in \{\mathtt{prog}, \mathtt{env}\}$ is a bookkeeping variable indicating whether this robot should be executing a program or environment transition.

For readability, we use the dot (".") notation to access components of a robot configuration L. For example, $L.M$ means accessing the local context M in the tuple L.

Black-box functions for environment transitions To define the executable $\mathbb{K}$ semantics of *Koord* applications, we have to provide executable descriptions for the environment transitions. The type of this executable object (f) is defined by *CPorts*, namely, $f : [CPorts \mapsto Val] \times \mathbb{R}_{\geq 0} \mapsto [CPorts \mapsto Val]$. That is, given old sensor and actuator values and a time point, f should return the new values for all sensor and actual ports. Depending on whether we have an explicit or a black-box model for f, the executable semantics will enable different types of analysis as we shall see in Section 4.

3.3 Semantics

We will describe only the interesting semantic rules of *Koord* above the event level. Rule LvarAssign and Rule SvarAssign showing the semantic rules for local and shared variable assignment respectively are provided as examples of statement level rules. Rule StmtSeq1 and StmtSeq2 show how a statement representing a sequence of statements is executed.

$$\frac{L.turn = \texttt{prog} \quad \wedge \text{“}Name\textbf{: pre: } Cond \textbf{ eff: } Body\text{”} \in Events \wedge [\![Cond]\!]_{S,L}}{\langle S, L, \oplus\rangle \rightarrow_{stmt} \langle S, L, Body\rangle} \textsc{SelectEvent}$$

$$\langle S, L, \oplus\rangle \rightarrow_{stmt} \langle S, L, \cdot\rangle \textsc{SkipEvent}$$

$$\langle S, (M, cp, \texttt{prog}), \cdot\rangle \rightarrow_{stmt} \langle S, (M, cp, \texttt{env}), \cdot\rangle \textsc{EndEvent}$$

$$\frac{\forall x \in Keys(S), M' = M[x \mapsto S[x]] \wedge cp' = f(cp, \delta)}{\langle S, (M, cp, \texttt{env})\rangle \rightarrow_{env} \langle S, (M', cp', \texttt{prog})\rangle} \textsc{RobotEnv}$$

Figure 7. Partial per robot semantic rules for *Koord*.

More details on statement and expression level semantics will be available in a future extended version of the paper.

Per robot semantics. First, we present the semantics of executing an event for each robot, which will help us discuss the semantics of the whole system. All rules for statement semantics are of type

$$\rightarrow_{stmt} \subseteq (\mathbb{S} \times \mathbb{L} \times (Stmt \cup \{\oplus, \cdot\})) \mapsto \wp(\mathbb{S} \times \mathbb{L} \times Stmt \cup \{\cdot\}),$$

where *Stmt* refers to the set of all possible statements allowed by *Koord* syntax. This relation takes as input a tuple of (1) a global context, (2) a robot configuration, and (3) a statement, and maps it to a set of such tuples.

The symbols '$\oplus$' and '$\cdot$' are not in *Koord* but internal syntactic structures. '$\oplus$' is to denote nondeterministic selection of events, and '$\cdot$' is to indicate an "empty" statement.

Rule SelectEvent in Figure 7 shows that any event may be executed when the precondition *Cond* is evaluated to true, and by replacing $\oplus$ with the event effect *Body*, it ensures only one event is selected and executed. The event effect is then executed following the semantics of each statement in *Body*. Rule SkipEvent allows the robot to skip the event completely. At the end of the event, the sequence of statements becomes empty '$\cdot$'. Rule EndEvent then makes sure the *turn* of the robot is set to env indicating that an environment transition will occur afterwards.

Similarly, we define the semantics of how each robot interacts with environment including other robots. The environment transition rule is of type

$$\rightarrow_{env} \subseteq (\mathbb{S} \times \mathbb{L}) \mapsto \wp(\mathbb{S} \times \mathbb{L}),$$

which takes a global context and a robot configuration as input. Rule RobotEnv simply states that the new local context M' is the old local context M updated with the global context S; thus ensuring that all robots have consistent shared variable values before the next program transition. New sensor readings cp' is then obtained by evaluating the black-box dynamics f with time δ. In an actual execution, the controller would run the program on hardware, whose sensor ports evolve for δ time between program transitions. This formalization allows the ports to behave arbitrarily over δ-transitions. Hence in verification, additional assumptions over the behavior of the sensor and actuator ports are needed. Finally, the *turn* of the robot is set back to prog.

Global semantics. With the event semantic for each robot, we can then define the execution for the distributed *Koord* program. The rewrite rule is a mapping from an initial system configuration to a set of configurations. It has the type

$$\rightarrow_G \subseteq \mathbb{C} \mapsto \wp(\mathbb{C}),$$

where $\mathbb{C}$ is the set of all possible global configurations.

$$\frac{\exists i \in \texttt{ID}, \langle S, L_i, \oplus\rangle \rightarrow_{stmt} \langle S', L_i', \cdot\rangle \quad \wedge L_i.turn = \texttt{prog} \wedge L_i'.turn = \texttt{env}}{(\{L_i\}, S, \tau, \texttt{prog}) \rightarrow_G (\{L_i'\}, S', \tau, \texttt{prog})} \textsc{EventTrans}$$

$$\frac{\forall i \in \texttt{ID}, L_i.turn = \texttt{env}}{(\{L_i\}, S, \tau, \texttt{prog}) \rightarrow_G (\{L_i\}, S, \tau, \texttt{env})} \textsc{EndProgTrans}$$

$$\frac{\forall i \in \texttt{ID}, \langle S, L_i\rangle \rightarrow_{env} \langle S, L_i'\rangle \quad \wedge L_i.turn = \texttt{env} \wedge L_i'.turn = \texttt{prog}}{(\{L_i\}, S, \tau, \texttt{env}) \rightarrow_G (\{L_i'\}, S, \tau + \delta, \texttt{prog})} \textsc{EnvTrans}$$

Figure 8. Global semantic rules for *Koord*.

Rule EventTrans expresses that starting from a global configuration $\boldsymbol{c} = (\{L_i\}, S, \tau, \texttt{prog})$, a robot i with the configuration L_i starts by selecting an enabled event, executes the event via a sequence of $\rightarrow_{stmt}$ rewrites, and sets its own *turn* to env at the end of the event execution. The system goes from a configuration $\boldsymbol{c}$ to $\boldsymbol{c}' = (\{L_i'\}, S', \tau, \texttt{prog})$, with possibly different robot configurations and global context depending on whether any statement executed resulted in writes to shared variables. The system can display nondeterministic behaviors arising from different robots executing their events in different orders. After all robots enter the env turn, rule EndProgTrans sets the global *turn* from prog to env indicating the end of program transition, and an environment transition will occur afterwards.

Rule EnvTrans shows the semantics of the system configuration after rule EndProgTrans. This rule synchronizes the environment transitions of each robot and ensure that the global time τ advances to $\tau + \delta$.

3.4 Synchronization and Consistency

Following our semantic rules in Section 3.3, careful readers would notice that all event transitions of *Koord* program takes *zero* time. The environment transitions however take δ time for the evolution of the sensor and actuator ports together with the update of the local context from the global context. In this section, we discuss how this semantic abstracts the behavior of a distributed cyber-physical system, and how our tool chain in [4] provides a faithful implementation of such abstraction.

To reiterate, the following are the timing requirements for rule EventTrans and EnvTrans: (a) an event transition takes *zero* time, (b) new values of controller ports are sampled at the end of each round (c) shared variables should reach consistent values within δ time, and (d) a global clock is used to synchronize each δ-time round. The first two requirements are achievable if the time taken to complete a program transition is negligible compared to δ, and δ can be a common multiple of the sampling intervals of all controller ports in use. These constraints are reasonable when computation and communication is comparatively much faster. For

Motion module as an example, our position sensor on each device publishes every 0.01 sec (100Hz) while the CPU on each drone is 1.4 GHz. If we set δ to be 0.01 sec, an event transition taking 10K CPU cycles is still less than 0.1% of δ.

Requirement (c) and (d) are common research questions in distributed computing with an extensive literature. A global clock can be achieved with existing techniques that synchronize all local clocks on robots. In [4], we use message passing to implement distributed shared memory for shared variables. We ensure that the time taken to propagate values through messages and reach consistency is smaller than δ, and the update is visible in the next round of program transitions for all robots. We therefore conclude our round based semantic with shared memory is a reasonable abstraction.

4 Verifying *Koord* programs

We have built the semantics of *Koord* in the $\mathbb{K}$ framework to enable decoupled analyses of platform-independent discrete part and the platform-dependent (dynamic) parts of distributed multi-robot systems. The *events* in an *Koord* program define the discrete computations in the system. The effect of a robot i executing event $e \in Events$ on a configuration $\boldsymbol{c} \in \mathbb{C}$, can be seen as a $\rightarrow_{stmt}$ application to $\langle \boldsymbol{c}.S, \boldsymbol{c}.L_i, Body\rangle$, where e is *"eventName:* **pre:** *Cond* **eff:** *Body"*.

4.1 Reachable configurations

Given a set of system configurations C, we define the following functions using the semantic rules of Section 3.3: (i) $Post_e(\boldsymbol{c}, i)$ returns the set of configurations obtained by robot i executing event $e \in Events$ from a configuration $\boldsymbol{c}$. (ii) $Post(C, i)$ returns the set of configurations obtained by robot i executing any event from a configuration in C. (iii) $Post(C, \vec{p})$ returns all configurations visited, when robots execute their events in the order $\vec{p}$, where $\vec{p}$ is a sequence of $p_i \in \mathtt{ID}$. (iv) $Post(C)$ is the union of $Post(C, \vec{p})$ over all orders $\vec{p}$. (v) $End_{\mathsf{prog}}(C)$ is the set of configurations reached from C *after* a program transition.

$$
\begin{aligned}
Post_e(\boldsymbol{c}, i) &:= \{\boldsymbol{c}' \mid [\![Cond]\!]_{\boldsymbol{c}.S, \boldsymbol{c}.L_i} \\
&\qquad \wedge \langle \boldsymbol{c}.S, \boldsymbol{c}.L_i, Body\rangle \rightarrow_{stmt} \langle \boldsymbol{c}'.S, \boldsymbol{c}'.L_i, \cdot\rangle\}, \\
Skip(\boldsymbol{c}, i) &:= \{\boldsymbol{c}' \mid \langle \boldsymbol{c}.S, \boldsymbol{c}.L_i, \cdot\rangle \rightarrow_{stmt} \langle \boldsymbol{c}'.S, \boldsymbol{c}'.L_i, \cdot\rangle\} \\
Post(C, i) &:= \bigcup_{\boldsymbol{c} \in C} \left(Skip(\boldsymbol{c}, i) \cup \bigcup_{e \in Events} Post_e(\boldsymbol{c}, i) \right), \\
Post(C, \vec{p}) &:= \begin{cases} \emptyset, & \text{if } \vec{p} = () \\ Post(Post(C, p_0), \vec{p}'), & \text{if } \vec{p} = (p_0, \vec{p}') \end{cases} \\
Post(C) &:= \bigcup_{\vec{p} \in perms(\mathtt{ID})} Post(C, \vec{p}), \\
End_{\mathsf{prog}}(C) &:= \{\boldsymbol{c} \in Post(C) \mid \forall i \in \mathtt{ID}, \boldsymbol{c}.L_i.turn = \mathsf{env}\}.
\end{aligned}
$$

In the above, a sequence $\vec{p} = (p_0, \vec{p}')$, is written as a concatenation of the first element p_0 and the suffix $\vec{p}'$. Also, $perms(\mathtt{ID})$ refers to the set of permutations of $\mathtt{ID}$.

Next, we define the configurations that are reached during and after an environment transition. Recall that environment transitions capture the evolution of the actuator ports over a time interval $[0, \delta]$—all other parts of the configuration remain unchanged. Our *Koord* semantics defines the environment transitions with a *parameter* which is a (possibly black-box) function that captures the dynamics of individual robots.[3] Given such a function f_i for each robot i, we define the function $traj : \mathbb{C} \times [0, \delta] \mapsto \mathbb{C}$ to represent the evolution of the system over a $[0, \delta]$ time interval. *traj* is constructed by simply update all controller ports *cp* of all robots with their f_i. That is,

$$
\boldsymbol{c}' = traj(\boldsymbol{c}, t) \Leftrightarrow \left(\begin{array}{l} \forall i \in \mathtt{ID}, \boldsymbol{c}'.L_i.cp = f_i(\boldsymbol{c}.L_i.cp, t) \\ \wedge\ \boldsymbol{c}'.L_i.M = \boldsymbol{c}.L_i.M \\ \wedge\ \boldsymbol{c}'.L_i.turn = \boldsymbol{c}.L_i.turn \\ \wedge\ \boldsymbol{c}'.S = \boldsymbol{c}.S \wedge \boldsymbol{c}'.\tau = \boldsymbol{c}.\tau \\ \wedge\ \boldsymbol{c}'.turn = \boldsymbol{c}.turn \end{array} \right)
$$

Notice those additional constraints making sure all other fields of $\boldsymbol{c}$ and $\boldsymbol{c}'$ stay the same. We will use $\wedge \ldots$ to skip these kind of constraints in the later sections for simplicity. The set of system configurations $Pt_{[t_1, t_2]}(C)$ reached in an interval $[t_1, t_2]$:

$$
Pt_{[t_1, t_2]}(C) := \{\boldsymbol{c}' \mid \exists \boldsymbol{c} \in C, t_1 \leq t \leq t_2, \boldsymbol{c}' = traj(\boldsymbol{c}, t)\}.
$$

The set of points reached at the end of an environment transition from C is denoted by $End_{\mathsf{env}}(C) := Pt_{[\delta, \delta]}(C)$.

Now to conform to our semantics, we carefully define the exact set of configurations reached right at the end of each *round* without transient configurations. A *frontier* set of configurations $End_{rnd}(C, n)$ represents those configurations that are reached from C *exactly when* n rounds are completed. Formally,

$$
End_{rnd}(C, n) := \begin{cases} C, & \text{if } n = 0 \\ End_{\mathsf{env}}(End_{\mathsf{prog}}(End_{rnd}(C, n-1))). \end{cases}
$$

Finally, given a set of configurations $C_0 \subseteq \mathbb{C}$, the set of all reachable states in n rounds is defined inductively:

$$
Reach(C_0, n) := \begin{cases} C_0 & \text{if } n = 0 \\ Reach(C_0, n-1) & \text{otherwise} \\ \quad \cup\ Post(End_{rnd}(C_0, n-1)) & \\ \quad \cup\ Pt_{[0, \delta]}(End_{\mathsf{prog}}(End_{rnd}(C_0, n-1))), & \end{cases}
$$

Notice that all transient configurations during both program (computed by *Post*) and environment (computed by $Pt_{[0, \delta]}$) transitions are included in *Reach*.

4.2 Decomposing invariance verification

An *invariant* of a *Koord* program is a predicate that holds in all reachable configurations. Invariant requirements can express safety, for instance, that no two robots are ever too close (Collision avoidance), or that robots always stay within a designated area (Geofencing). Formally,

[3] For different platforms, this function could be defined in closed form, as solutions of differential equations, or in terms of a numerical simulator.

Definition 4.1. An invariant *inv* is a predicate (Boolean valued function) over a configuration $\boldsymbol{c}$ such that, given a set of initial configurations of the system C_0,

$$\forall n \in \mathbb{N}, \forall \boldsymbol{c} \in Reach(C_0, n), [\![inv]\!]_{\boldsymbol{c}},$$

where $[\![inv]\!]_{\boldsymbol{c}}$ represents evaluating *inv* over $\boldsymbol{c}$.

Definition 4.2. A predicate *inv* is an *inductive invariant* of a system if given a set of initial configurations of the system C_0, the following proof obligations (POs) hold:

$$\forall \boldsymbol{c}_0 \in C_0, [\![inv]\!]_{\boldsymbol{c}_0} \qquad (Base)$$

$$\forall \boldsymbol{c} \in \mathbb{C}, [\![inv]\!]_{\boldsymbol{c}} \Rightarrow \forall \boldsymbol{c}' \in Reach(\{\boldsymbol{c}\}, 1), [\![inv]\!]_{\boldsymbol{c}'} \qquad (1)$$

That is, *inv* holds in the initial configuration(s) (PO (*Base*)), and *inv* is preserved by both platform-independent discrete program transitions and the platform-dependent environment transitions (PO (1)). It is straightforward to prove that an inductive invariant is an invariant.

Our verification strategy for user-specified (inductive) invariants is to dischare the proof obligations. PO (*Base*) is usually trivial. Therefore, we focus on PO (1). The *Koord* semantics enables us to *decouple* the environment and program transitions in *Reach*, and analyze each separately. PO (1) can be restated by expanding *Reach* and End_{rnd} as $\forall \boldsymbol{c} \in \mathbb{C}$,

$$[\![inv]\!]_{\boldsymbol{c}} \Rightarrow \forall \boldsymbol{c}' \in Post(\{\boldsymbol{c}\}), [\![inv]\!]_{\boldsymbol{c}'} \qquad (2)$$

$$[\![inv]\!]_{\boldsymbol{c}} \Rightarrow \forall \boldsymbol{c}' \in Pt_{[0,\delta]}(End_{\mathsf{prog}}(\{\boldsymbol{c}\})), [\![inv]\!]_{\boldsymbol{c}'}. \qquad (3)$$

As in other concurrent systems, a major bottleneck in computing *Post* for PO (2) is the required enumeration of all $\vec{p} \in perms(\mathtt{ID})$ permutations for all robots with reads/writes to shared contexts. We therefore seek for a stronger and easier to prove proof obligation using the lemma below:

Lemma 4.3. *Given any inductive predicate φ, for any configuration c satisfying φ, the following always holds*

$$\left(\bigwedge_{i \in \mathtt{ID}} \bigwedge_{e \in Events} \forall \boldsymbol{c}' \in Post_e(\boldsymbol{c}, i), [\![\varphi]\!]_{\boldsymbol{c}'}\right) \Rightarrow \forall \boldsymbol{c}' \in Post(\{\boldsymbol{c}\}), [\![\varphi]\!]_{\boldsymbol{c}'}$$

The proof follows from expanding the definition of *Post* and inducting on each event sequence. As φ is preserved before and after every event transition $Post_e$ of every robot, the order of robot events do not violate φ. With Lemma 4.3, we strengthen and rewrite PO (2) as $\forall \boldsymbol{c}, \boldsymbol{c}' \in \mathbb{C}$,

$$\bigwedge_{i \in \mathtt{ID}} \bigwedge_{e \in Events} [\![inv]\!]_{\boldsymbol{c}} \wedge \boldsymbol{c}' \in Post_e(\boldsymbol{c}, i) \Rightarrow [\![inv]\!]_{\boldsymbol{c}'} \qquad (4)$$

which no longer requires enumeration of all permutations. This is the main reason how our synchronous model of execution help verification scale.

We now discuss our approach to discharge PO (3). To further decouple program and environment transitions, we expand $Pt_{[0,\delta]}$ and rewrite PO (3) as $\forall \boldsymbol{c}, \boldsymbol{c}', \boldsymbol{c}'' \in \mathbb{C}$,

$$\begin{aligned}&([\![inv]\!]_{\boldsymbol{c}} \wedge \ \boldsymbol{c}' \in End_{\mathsf{prog}}(\{\boldsymbol{c}\}) \\ &\quad \wedge \forall t \in [0,\delta], \boldsymbol{c}'' = traj(\boldsymbol{c}', t)) \Rightarrow [\![inv]\!]_{\boldsymbol{c}''}.\end{aligned} \qquad (5)$$

PO (5) requires reasoning about the dynamic behavior of *traj* during environment transitions, and it is a challenging research problem by itself. We introduce *port assumptions* to abstract away the continuous dynamic behavior.

Definition 4.4. Given a configuration $\boldsymbol{c}'$ and *traj*, a *port assumption* $A(\cdot,\cdot)$ is a predicate on $\mathbb{C} \times \mathbb{C}$ if $\forall \boldsymbol{c}', \boldsymbol{c}'' \in \mathbb{C}$,

$$(\forall t \in [0,\delta], \boldsymbol{c}'' = traj(\boldsymbol{c}', t)) \Rightarrow A(\boldsymbol{c}', \boldsymbol{c}''). \qquad (PAsm)$$

Port assumptions allow users to over-approximate *traj* and prove the invariant at hand. PO (*PAsm*) can be validated with our *Koord* simulator or with other specialized tools for continuous dynamics (see Section 4.3). Further, we know by definition $End_{\mathsf{prog}}(\{\boldsymbol{c}\}) \subseteq Post(\{\boldsymbol{c}\})$, we can apply Lemma 4.3 in a similar way. Hence, with PO (*PAsm*) and Lemma 4.3, we can merge PO (4) and PO (5) and strengthen as $\forall \boldsymbol{c}, \boldsymbol{c}', \boldsymbol{c}'' \in \mathbb{C}$,

$$\bigwedge_{i \in \mathtt{ID}} \bigwedge_{e \in Events} [\![inv]\!]_{\boldsymbol{c}} \wedge \boldsymbol{c}' \in Post_e(\boldsymbol{c}, i) \wedge A(\boldsymbol{c}', \boldsymbol{c}'') \Rightarrow [\![inv]\!]_{\boldsymbol{c}''} \quad (Ind)$$

where we can use our $\mathbb{K}$ symbolic execution semantics to construct the symbolic post configuration to represent $\boldsymbol{c}' \in Post_e(\boldsymbol{c}, i)$ for each event. Notice that PO (*Ind*) allows us to reason in per event fashion as well as per robot fashion.

4.3 Validating port assumptions: reachability analysis

Over three decades of research on verification of complex dynamical and hybrid systems [2, 34] has led to the creation of a powerful toolbox for linear [5, 19], nonlinear [11, 16, 18], and black-box systems [17]. Depending on the type and availability of the platform models, these tools can be used for discharging the port assumptions. Here, for the sake of completing the picture, we briefly mention how the traces from the *Koord* simulator together with the DryVR tool [17], could be used to check port assumptions for various platforms or find counterexamples.

DryVR uses numerical simulations to learn the sensitivity of the trajectories of the vehicle to changes in initial conditions, with a certain confidence level. Then it uses this sensitivity and additional simulations to either prove the given invariant (in our case the port assumption) or find a counter-example. Under certain robustness assumptions, this process is also guaranteed to terminate. We used *Koord* simulator to generate traces of a vehicle (and quadcopter) moving from a set of initial conditions to a target waypoint. From these traces, DryVR computes the reachable states (see Figure 9). Notice that for the same relative distance between the initial position and the target, the quadcopter has a larger reachset than the vehicle because the former overshoots.

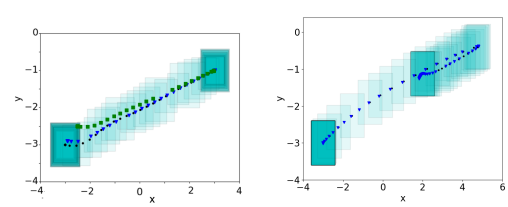

Figure 9. *Left*: Reachsets for car. *Right*: Reachtube for drone

5 Case studies

We demonstrate the expressive power and verification capability of *Koord* on three case studies. In each case study, we describe the two key proof obligations PO (*Ind*) and PO (*PAsm*) as well as the port assumptions in use. For each case, we also show experiment results for proving/disproving the POs.

5.1 LineForm: Distributed formation control

In this section, we revisit the LineForm program in Section 2. As mentioned in Section 4.2, the symbolic post configuration generated by $\mathbb{K}$ is representing a set of *global configurations*. For readability, we again present simplified formulas. Following the notation in PO (*Ind*) and PO (*PAsm*), variables and *primed copies* represents the variables in $\boldsymbol{c}$, $\boldsymbol{c}'$, and $\boldsymbol{c}''$ respectively.

Symbolically executing the event *TargetUpdate* (for robot i) generates the constraint:

$$E := \begin{pmatrix} \neg(i = \texttt{N_SYS} - 1 \vee i = 0) \\ \wedge\ M.tgt'_i = (x[i-1] + x[i+1])/2 \wedge x'[i] = M.pos_i \\ \wedge\ \ldots \end{pmatrix}$$

Notice that it starts with the precondition and $M.tgt_i$ and $x[i]$ are updated according to the effect. We omit the rest of the formula that ensures the values of unmodified variables are unchanged such as $M.pos'_i = M.pos_i$ and $x'[j] = x[j]$ for $j \neq i$. Additionally, a more accurate version of Assumption 1 in Section 2 is

Port Assumption 2.

$$A := M.pos''_i \in rect(M.pos'_i, M.tgt'_i) \wedge \ldots$$

where the rest of the formula ensuring unchanged values is omitted. Consequently, PO (*PAsm*) becomes

Proof Obligation 1.

$$(\forall t \in [0, \delta], M.pos''_i = f(M.pos'_i, M.tgt'_i, t)) \Rightarrow A$$

The invariant for the current configuration ($[\![inv]\!]_{\boldsymbol{c}}$) is

$$I := M.pos_i \in rect(x_{min}, x_{max}) \wedge x[i] \in rect(x_{min}, x_{max})$$

and the invariant over primed configuration ($[\![inv]\!]_{\boldsymbol{c}''}$) is

$$I'' := M.pos''_i \in rect(x_{min}, x_{max}) \wedge x''[i] \in rect(x_{min}, x_{max})$$

Because there is only one event, PO (*Ind*) for LineForm then becomes

Proof Obligation 2. $\bigwedge_{i \in \texttt{ID}} I \wedge E \wedge A \Rightarrow I''$

Table 1 summarizes the verification of these constraints on systems with different N_SYS.

We can define a *WI*, a weaker invariant which simply constrains the position of each robot, and doesn't constrain the shared $x[i]$,

$$WI := M.pos_i \in rect[x_{min}, x_{max}]$$

Table 2 shows the results we obtained without the proof obligations as assumptions, and this weaker invariant on a system of robots moving in 2D. The verification procedure only tells us that the invariant is not inductive, it doesn't tell us whether the invariant doesn't hold, which is why we don't know whether the system is safe w.r.t the proposed invariant.

N_SYS	dim	T_K (s)	T_V (s)	Safe	N_SYS	dim	T_K (s)	T_V (s)	Safe
3	1	4.90	9.09	✓	5	2	5.60	18.91	✓
3	2	4.19	10.13	✓	5	3	5.42	20.30	✓
4	1	4.79	12.21	✓	10	1	10.92	32.34	✓
4	2	5.28	12.49	✓	10	2	10.96	32.42	✓
4	3	5.06	12.77	✓	10	3	11.34	33.61	✓
5	1	4.91	18.46	✓	15	1	12.23	53.89	✓

Table 1. Summary of semantics based verification for LineForm. N is N_SYS, T_K is the symbolic post computation time in $\mathbb{K}$, T_V is the time taken for construction of constraints and verification in Z3. Robots moving along a line are represented by **dim** = 1, along a plane by **dim** = 2, and in a 3D space by **dim** = 3.

N_SYS	constraint	T_K (s)	T_V (s)	Safe
15	$E \wedge I \Rightarrow I'$	13.06	23.03	*Unknown*
15	$E \wedge WI \Rightarrow WI'$	9.92	18.26	*Unknown*
15	$E \wedge A \wedge WI \Rightarrow WI'$	11.24	32.64	*Unknown*

Table 2. Verification summary for weaker invariants on LineForm. The **constraint** column displays the induction hypothesis used in the verification.

```
using Motion:
  actuators:
    List⟨Point⟩ path
  sensors:
    Point position
    bool reached
    PathPlanner planner
local:
  bool on_task = s
  List⟨Point⟩ curr_path
  Task cur_task

allread:
  List⟨Point⟩ shared_paths[N_SYS]
allwrite:
  List⟨Task⟩ all_tasks

Complete:
pre: on_task and Motion.reached
eff: on_task=False
shared_paths[pid]=[Motion.position]
Assign:
  pre: !on_task
  eff:
    if len(all_tasks) == 0:
      stop
    else: atomic:
      for t in all_tasks:
        curr_path=Motion.planner(t.target)
        if pathIsClear(shared_paths, \
                 curr_path, pid):
          on_task=True
          cur_task=t
          break
      if on_task:
        all_tasks.remove(cur_task)
        shared_paths[pid]=curr_path
        Motion.path=curr_path
      else:
        shared_paths[pid]=[Motion.position]
```

Figure 10. *Koord* code for robot i for the Distributed Task Allocation

5.2 Task: Distributed task allocation

Task in Figure 10 is a simple *Koord* program to solve distributed task allocation problem. The problem statement is as follows: Given a set of (possibly heterogeneous) robots, a safety distance $\epsilon > 0$, and a fixed sequence of points (tasks) $list = x_1, x_2, \ldots \in \mathbb{R}^3$, there are following two requirements: (a) every unvisited x_i in the sequence is *visited* exactly by one robot and (b) no two robots ever get closer than ϵ. In this case study, we only prove requirement (b) for Task.

Task consists of two events (1) *Assign*, in which each robot looks for an unassigned task x from *list*; if there is a clear path to x then the robot assigns itself the task x, set the actuator port *Motion.path*, and shares its path with all other robots thru *shared_path*. Otherwise, it shares its position as the path. (2) *Complete*, which checks whether an robot has visited its assigned task. A path here is a list of points that a robot visits in sequence. The *Motion* module drives the robot along

a path, as directed by the position value set at its actuator port *Motion.path*. The sensor port *Motion.planner* returns a path to the target of an unassigned task, and a (user-defined) function called *pathIsClear* is used to determine whether the currently planned path is within ϵ distance of any path in *shared_path*.

Suppose there is a function taking two paths as input $clear_\epsilon : List\langle Point\rangle \times List\langle Point\rangle \mapsto bool$, it returns true only if the minimum distance between the two paths is greater than ϵ. We restate requirement (b) as:

Invariant 2. *No two robots ever get closer than ϵ.*

$$\forall i, j, \in \texttt{ID}, clear_\epsilon(shared_path[i], shared_path[j])$$

Computing the $clear_\epsilon$ function involves nested loops over the length of each path, then computing the minimum distance between each path segment. *pathIsClear* further has to iterate over all shared paths and check via $clear_\epsilon$. Proving invariant over these loops is by itself a well studied and difficult research problem. To mitigate this problem, we instruct our symbolic execution to treat $clear_\epsilon$ and *pathIsClear* as *uninterpreted functions*, and we introduce *function summary* for these uninterpreted functions similar to port assumptions.

Definition 5.1. A *function summary* $F(x, y)$ for an uninterpreted function $f(x)$ is a predicate which we can prove the following proof obligation:

$$\forall x, F(x, f(x)) \qquad (FSum)$$

where x can be extended according to the arity of f. Verification and generation of good function summaries is extensively discussed and widely used in software verification. We believe writing a good function summary requires strong domain knowledge in particular robot devices as well as the problem to be solved. Here we only showcase a function summary *PIC* for *pathIsClear*:

Function Summary 1.

$$PIC(sp, cp, i, y) := \forall j \in \texttt{ID}, j \neq i \wedge \neg clear_\epsilon(sp[j], cp) \Rightarrow \neg y$$

The function summary simply says, if my current path *cp* is not more than ϵ distance to any path $sp[j]$ shared by other robots, the output of $y = pathIsClear(sp, cp, i)$ should be false,[4] and PO (*FSum*) becomes:

Proof Obligation 3.

$$\forall sp, cp, i, PIC(sp, cp, i, pathIsClear(sp, cp, i))$$

For constructing the symbolic set of configurations, we use a list with four tasks signified by $\{t_1, t_2, t_3, t_4\}$ so that the symbolic execution terminates. The for-loop iterating through the task list is unrolled into a sequence of (nested) *if-else* statements. For simplicity, we show the symbolic post event configuration of the *Assign* event for only one execution when robot i picks t_1:

$$\begin{aligned} E_{t_1} := {} & \neg on_task_i \wedge on_task_i' \\ & \wedge\ curr_path_i' = M.planner(t_1.target) \\ & \wedge\ PIC(shared_path, curr_path_i', i, \texttt{True}) \\ & \wedge\ shared_path'[i] = curr_path_i{}' \\ & \wedge\ M.path_i' = shared_path'[i] \wedge \ldots \end{aligned}$$

Notice how we can use *PIC* to summarize *pathIsClear*. Similarly, we get E_{t_2}, E_{t_3} and E_{t_4} for other execution paths choosing corresponding tasks. When none of the tasks is picked, the post event configuration generated is

$$E_{none} := \neg on_task_i \wedge shared_path'[i] = [M.pos_i] \wedge \ldots$$

For abstracting the movement of robots, an robot should move closely ($\neg clear_\epsilon$) along its *Motion.path* actuator whose value is denoted by $M.path$ until it finishes traversing the path. For simplicity, we denote *Motion.reached* by $M.reached$.

Port Assumption 3.

$$A := (\neg M.reached_i' \Rightarrow \neg clear_\epsilon(M.pos_i'', M.path_i'')) \wedge \ldots$$

Similarly, we write down the PO (*PAsm*) for discharging Assumption 3.

Proof Obligation 4.

$$\forall t \in [0, \delta],$$
$$(M.pos'', M.reached'') = f(M.pos', M.reached', M.path', t) \Rightarrow A$$

Finally, Invariant 2 for the current configuration for robot i reduces to

$$I := \forall j \in \texttt{ID}, j \neq i \Rightarrow clear_\epsilon(shared_path[j], shared_path[i]),$$

and we can easily derive I'' using $shared_path''[j]$ instead. The proof obligation PO (*Ind*) for event *Assign* then becomes

Proof Obligation 5. $\bigwedge_{i \in \texttt{ID}} I \wedge (\bigvee_{t_j} E_{t_j} \vee E_{none}) \wedge A \Rightarrow I''$

The induction hypothesis for event *Complete* is generated similarly (omitted here), and the overall proof obligation is a conjunction of the two. Table 3 summarizes the verification of these constraints with different number of robots.

Benchmark	N_SYS	T_K (s)	T_V (s)	Safe
Task	3	9.90	10.6	✓
Task	4	9.79	11.78	✓
Task	5	9.91	14.92	✓
Task	10	12.92	18.34	✓

Table 3. Summary of semantics based verification for Task. T_K is the symbolic post computation time in $\mathbb{K}$, T_V is the time taken for generation of constraints and verification in Z3 and N_SYS is the number of robots in the system.

5.3 Case Study: Mapping

For our last case study, we discuss the distributed grid mapping problem and our Mapping algorithm in *Koord* for the problem. The distributed grid mapping problem requires a set of robots to collaboratively mark the position of static *obstacles* within a given area D quantized by a *grid*, which any robot should avoid while moving in D. This problem is a

[4]The index i in the *pathIsClear* function indicates that the previously shared path of the robot i shouldn't be considered in the computation.

simplified version of the distributed Simultaneous Localization and Mapping problem, a classical problem in robotics research. The difference comes with the assumption that the robots know their *global coordinates* within the area of deployment, and only attempt to map static obstacles within this area. Further, the only sensors available for sensing obstacles are LIDAR based, and the robots are constrained to move in a 2D space.

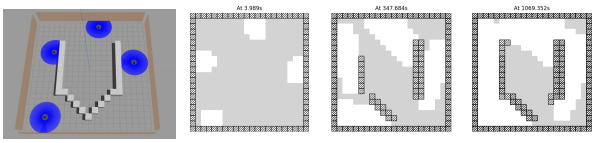

Figure 11. Four cars in U-shape world in simulator (*Left*). Visualization of the global map at three different time stamps (*Right*)

Mapping algorithm works in the following manner. Each robot constructs a *local grid map* over D using sensors, and updates it using information from other robots shared via a *global grid map*. In Figure 12, the *MotionWithScan* module provides a *pscan* sensor used to read the LIDAR scan of the actual robot. The other sensors and actuators *position, reached, planner, path* have the same functionality as that in the *Motion* module. The shared allwrite variable *map* is used to construct a shared map of obstacles within the domain D, and has type *GridMap*, which is a 2-D array representing a grid over D. The local variable *localMap* represents each robot's *local* knowledge of the domain D, and has the same type as D. There are three *events*: *NewPoint, LUpdate*, and *GUpdate*. A robot executing the *NewPoint* event, finds an unoccupied point to move to using a user defined function *pickFrontierPos* and plans a path to it using *MotionWithScan.planner*. It then updates its *localMap* from the shared variable *map*. The *LUpdate* event updates the *localMap* with scanned sensor data while the robot is in motion, and the *GUpdate* event updates the shared *map* with the updated *localMap* information corresponding to the scanned data.

A correctness requirement for Mapping is to ensure that, at any time, the global grid map *map* and all local maps $localMap_i$ should be consistent with the ground truth. To formally define this requirement as an invariant, we first introduce some auxiliary functions. Each grid map essentially maps a set $Q \subseteq D$ to 0 or 1 based on its occupancy, where Q is a quantized representation of D. For example, we can use $D = \mathbb{R}^2$ and $Q = \mathbb{Z}^2$ in a 2D world. Suppose there is a *ground truth* function $world_Q : Q \mapsto \{0, 1\}$ that gives the actual occupancy of obstacles in this quantized area. That is, $\forall q \in Q$,

$$world_Q(q) = \begin{cases} 1, & \text{if } q \text{ is occupied} \\ 0, & \text{otherwise} \end{cases}$$

We can define a function $chk : GridMap \mapsto \mathbb{B}$ such that $chk(g) := \forall q \in Q, (g(q) = 1) \Rightarrow (world_Q(q) = 1)$ to make sure the detected occupied grids are consistent. We then can formally define the invariant as:

```
using MotionWithScan
  sensors:
    Point position
    List⟨Point, Scan⟩ pscan
    bool reached
    PathPlanner planner
  actuators:
    List⟨Point⟩ path

allwrite:
  GridMap map

#omitting initialization
local:
  GridMap localMap
  Point target
  bool on_path = True
  List⟨Grid⟩ obstacles

GUpdate:
  pre MotionWithScan.reached
  eff: atomic:
    map = merge(map, localMap)
    on_path = False
NewTarget:
  pre !on_path
  eff:
    target = pickFrontierPos(map, \
                 MotionWithScan.position)
    obstacles = findObs(map)
    MotionWithScan.path = \
      MotionWithScan.planner(target, obstacles)
    if MotionWithScan.path != []:
      on_path = True
    else:
      on_path = False
    localMap = map

LUpdate:
  pre on_path and !MotionWithScan.reached
  eff:
    for p, s in MotionWithScan.pscan:
      localMap = merge(localMap, \
                   scanToMap(p, s))
```

Figure 12. *Koord* code for Distributed Mapping Application Mapping

Invariant 3. $I := \bigwedge_{i \in \mathtt{ID}} chk(localMap_i) \wedge chk(map)$

For the *NewPoint* event, the post event configuration for robot i is

$$E_{NP} := \neg on_path \wedge localMap_i' = map \wedge map' = map \wedge \ldots$$

Here we skip the updates of all other variables because they do not appear in Invariant 3. Surprisingly, the port assumption A for this case simply requires the program variables unchanged.

Port Assumption 4.

$$A := map'' = map' \wedge \bigwedge_{i \in \mathtt{ID}} localMap_i'' = localMap_i'$$

We skip the proof obligation PO (*PAsm*) for A because it is trivial. I is exactly as Invariant 3, and similar I'' is the primed version. PO (*Ind*) for the *NewPoint* event is

Proof Obligation 6. $\bigwedge_{i \in \mathtt{ID}} I \wedge E_{NP} \wedge A \Rightarrow I''$

For the event *GUpdate*, a function summary for the *merge* function is required. We simply provide a summary *MERGE* stating that the *merge* function returns a map satisfying *chk* when given two maps satisfying *chk*.

Function Summary 2.

$$MERGE(m_1, m_2, m') := chk(m_1) \wedge chk(m_2) \Rightarrow chk(m')$$

The specific proof obligation then is to prove the following:

Proof Obligation 7.

$$\forall m1, m2, MERGE(m1, m2, merge(m1, m2))$$

We then get the post event configuration for robot i as

$$E_G := \begin{array}{l} M.reached \\ \wedge\ MERGE(map, localMap_i, map') \\ \wedge \neg on_path' \wedge\ localMap_i' = localMap_i \wedge \ldots \end{array}$$

Benchmark	robots(N)	T_K (s)	T_V (s)	Safe
DMap	3	9.23	14.53	✓
DMap	4	9.33	19.25	✓
DMap	5	9.19	24.30	✓
DMap	10	9.31	59.81	✓

Table 4. Summary of semantics based verification for DMap. T_K is the symbolic post computation time in $\mathbb{K}$, T_V is the time taken for generation of constraints and verification in Z3 and *robots*(N) is the number of robots in the system.

I, I'', and A are the same as those for the *NewPoint* event. The overall constraint for this event along with its precondition is therefore:

Proof Obligation 8. $\bigwedge_{i \in \mathtt{ID}} I \wedge E_G \wedge A \Rightarrow I''$

We omit the discussion for *LUpdate* in the interest of space, as it follows along the lines of the previous constraints. It requires an additional proof obligation on the *scanToMap* function and the consistency of the sensor data $M.pscan$ with the ground truth, along with unrolling the *for-loop* to the number of sensor observations. The overall induction proof obligation is a conjunction of the three.

Table 4 summarizes the verification of these constraints on systems of different `N_SYS` .

6 Related work

Early domain specific languages for robotics were proprietary and tied to specific platforms. See [30] for a detailed survey. With the lowering hardware costs and increasing popularity, there is a growing interest in open and portable frameworks and languages [8, 32, 40, 41]. *Robot Operating System (ROS)* [35] is the predominant member in this category. At its core, ROS supports a publish-subscribe-based communication and the ROS community has built drivers for numerous hardware components. Our implementation of the *Koord* abstractions for the quadcopter and vehicle platforms use ROS just like thousands of other robotics products and projects. The Table above gives a summary of robotics languages that have been deployed on hardware.

Framework /system	Dist. Sys.	Hetero-geneous	Sim	Prog. Lang.	Compiler	V&V
ROSBuzz [38]	✓	✓	✓	Buzz	✓	
PythonRobotics		✓	✓	Python		
PyRobot [28]		✓	✓	Python		
MRPT [6]		✓		C++		
Robotarium [31]		✓	✓	Matlab		
Drona [14]	✓		✓	P [13]	✓	✓
Live [10]		✓		LPR	✓	
Koord	✓	✓	✓	*Koord*	✓	✓

ROSBuzz [38] supports the Buzz language, which doesn't provide abstractions like *Koord* for path planning and shared variables. The Live Robot Programming language [10] provides abstractions in terms of nested state machines and allows the program to be changed while running. It does not support robot ensembles. Programming systems using the shared memory paradigm have been developed for several distributed computing systems [1, 9, 12, 24, 29]. A position paper [22] proposed combining shared memory with physical interactions in a high-level language. This paper presents a full language, its formalization, and the proof system that combines those abstractions. P [13] and PSync [15] are DSLs for asynchronous partially distributed systems, but cyber-physical interactions are not supported. P has been integrated into the DRONA framework [14] and the latter has very similar objectives to our work, but the approaches and solutions are different. In brief, DRONA abstractions, like conflict-free path planning, are more concrete and dynamics-dependent, than *Koord*'s abstractions. In fact, our Task application implements something similar to the distributed plan generator which is a built-in feature for DRONA. On the other hand, *Koord*'s port interfaces allow portability across arbitrary planners.

In [4] we present the overview of the whole software stack on which Koord runs, with the focus on deploying Task application (2nd case study) on vehicle and quadcopter hardware[5]. In contrast, this paper presents the formal description of *Koord* language and the key language and verification challenges.

7 Conclusions and Future Work

Our vision is to provide a programming methodology to enable programming safe distributed cyber-physical applications without the need of complete domain expertise in all related areas such as control theory, robotics motion control, and network protocols. To this end, we demonstrated how *Koord* application developers can write succinct multi-robot applications involving distributed coordination, different types of sensing and actuation, and path planning in three case studies, each of which requires only preliminary knowledge in shared memory and basic concurrency control via atomic blocks and assumptions on sensor and actuator ports of controllers. V&V engineers with deep understanding in distributed computing are able to focus on formally analyzing invariant properties of *Koord* programs via symbolic execution, and roboticists can validate the feasibility of assumptions by examining rigorously defined proof obligations.

We acknowledge the fact that our port assumption based abstractions may not cover various vastly different types of robots. $\mathbb{K}$ semantics framework can allow us to extend our language to tailor to specific robot types on demand while retaining the same framework for formal analysis. We also plan to extend this work to include specification and verification of progress properties under fairness constraints for *Koord* applications.

References

[1] Sarita V. Adve and Kourosh Gharachorloo. 1996. Shared memory consistency models: A tutorial. *IEEE Computer* 29 (1996), 66–76.
[2] Rajeev Alur. 2015. *Principles of Cyber-Physical Systems.* The MIT Press.

[5] We have sent this paper to the PC chair.

[3] Hagit Attiya and Jennifer Welch. 2004. *Distributed Computing: Fundamentals, Simulations and Advanced Topics.* John Wiley & Sons, Inc., USA.

[4] Anonymous Author. 2019. Anonymized Paper. In *ICRA 2020 submission.* 6 pages.

[5] Stanley Bak and Parasara Sridhar Duggirala. 2017. Hylaa: A tool for computing simulation-equivalent reachability for linear systems. In *Proceedings of the 20th International Conference on Hybrid Systems: Computation and Control.* ACM, 173–178.

[6] José-Luis Blanco. 2009. *Contributions to Localization, Mapping and Navigation in Mobile Robotics.* Ph.D. Dissertation. PhD. in Electrical Engineering, University of Malaga. http://www.mrpt.org/Paper:J.L._Blanco_Phd_Thesis

[7] Timo Blender, Thiemo Buchner, Benjamin Fernandez, Benno Pichlmaier, and Christian Schlegel. 2016. Managing a Mobile Agricultural Robot Swarm for a seeding task. In *IECON 2016-42nd Annual Conference of the IEEE Industrial Electronics Society.* IEEE, 6879–6886.

[8] Brandon Bohrer, Yong Kiam Tan, Stefan Mitsch, Magnus O. Myreen, and André Platzer. 2018. VeriPhy: Verified Controller Executables from Verified Cyber-physical System Models. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2018).* ACM, New York, NY, USA, 617–630. https://doi.org/10.1145/3192366.3192406

[9] Brad Calder, Ju Wang, Aaron Ogus, Niranjan Nilakantan, Arild Skjolsvold, Sam McKelvie, Yikang Xu, Shashwat Srivastav, Jiesheng Wu, Huseyin Simitci, Jaidev Haridas, Chakravarthy Uddaraju, Hemal Khatri, Andrew Edwards, Vaman Bedekar, Shane Mainali, Rafay Abbasi, Arpit Agarwal, Mian Fahim ul Haq, Muhammad Ikram ul Haq, Deepali Bhardwaj, Sowmya Dayanand, Anitha Adusumilli, Marvin McNett, Sriram Sankaran, Kavitha Manivannan, and Leonidas Rigas. 2011. Windows Azure Storage: A Highly Available Cloud Storage Service with Strong Consistency. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles (SOSP '11).* ACM, New York, NY, USA, 143–157. https://doi.org/10.1145/2043556.2043571

[10] Miguel Campusano and Johan Fabry. 2017. Live Robot Programming: The Language, its Implementation, and Robot API Independence. *Science of Computer Programming* 133 (2017), 1 – 19.

[11] Xin Chen, Erika Ábrahám, and Sriram Sankaranarayanan. 2013. Flow*: An Analyzer for Non-linear Hybrid Systems. In *Computer Aided Verification*, Natasha Sharygina and Helmut Veith (Eds.). Lecture Notes in Computer Science, Vol. 8044. Springer Berlin Heidelberg, 258–263.

[12] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels. 2007. Dynamo: Amazon's Highly Available Key-value Store. In *Proceedings of Twenty-first ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07).* ACM, New York, NY, USA, 205–220. https://doi.org/10.1145/1294261.1294281

[13] Ankush Desai, Vivek Gupta, Ethan Jackson, Shaz Qadeer, Sriram Rajamani, and Damien Zufferey. 2013. P: Safe Asynchronous Event-driven Programming. *SIGPLAN Not.* 48, 6 (June 2013), 321–332. https://doi.org/10.1145/2499370.2462184

[14] Ankush Desai, Indranil Saha, Jianqiao Yang, Shaz Qadeer, and Sanjit A Seshia. 2017. Drona: A framework for safe distributed mobile robotics. In *2017 ACM/IEEE 8th International Conference on Cyber-Physical Systems (ICCPS).* IEEE, 239–248.

[15] Cezara Drăgoi, Thomas A. Henzinger, and Damien Zufferey. 2016. PSync: A Partially Synchronous Language for Fault-tolerant Distributed Algorithms. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '16).* ACM, New York, NY, USA, 400–415. https://doi.org/10.1145/2837614.2837650

[16] Parasara Sridhar Duggirala, Sayan Mitra, and Mahesh Viswanathan. 2013. Verification of Annotated Models from Executions. In *EMSOFT.*

[17] Chuchu Fan, Bolun Qi, Sayan Mitra, and Mahesh Viswanathan. 2017. DryVR: Data-driven verification and compositional reasoning for automotive systems. In *Computer Aided Verification (CAV).*

[18] Chuchu Fan, Bolun Qi, Sayan Mitra, Mahesh Viswanathan, and Parasara Sridhar Duggirala. 2016. Automatic Reachability Analysis for Nonlinear Hybrid Models with C2E2. In *Computer Aided Verification - 28th International Conference, CAV 2016, Toronto, ON, Canada, July 17-23, 2016, Proceedings, Part I.* 531–538. https://doi.org/10.1007/978-3-319-41528-4_29

[19] Goran Frehse, Colas Le Guernic, Alexandre Donzé, Scott Cotton, Rajarshi Ray, Olivier Lebeltel, Rodolfo Ripado, Antoine Girard, Thao Dang, and Oded Maler. 2011. SpaceEx: Scalable Verification of Hybrid Systems. In *CAV (Lecture Notes in Computer Science)*, Ganesh Gopalakrishnan and Shaz Qadeer (Eds.), Vol. 6806. Springer, 379–395.

[20] David Gauthier, Paul Freedman, Gregory Carayannis, and Alfred Malowany. 1987. Interprocess communication for distributed robotics. *IEEE Journal on Robotics and Automation* 3, 6 (1987), 493–504.

[21] Mario Gerla, Eun-Kyu Lee, Giovanni Pau, and Uichin Lee. 2014. Internet of vehicles: From intelligent grid to autonomous cars and vehicular clouds. In *2014 IEEE world forum on internet of things (WF-IoT).* IEEE, 241–246.

[22] Ritwika Ghosh, Sasa Misailovic, and Sayan Mitra. 2018. Language Semantics Driven Design and Formal Analysis for Distributed Cyber-Physical Systems: [Extended Abstract]. In *Proceedings of the 2018 Workshop on Advanced Tools, Programming Languages, and PLatforms for Implementing and Evaluating Algorithms for Distributed Systems (ApPLIED '18).* ACM, New York, NY, USA, 41–44. https://doi.org/10.1145/3231104.3231958 event-place: Egham, United Kingdom.

[23] Ge Guo and Wei Yue. 2012. Autonomous platoon control allowing range-limited sensors. *IEEE Transactions on vehicular technology* 61, 7 (2012), 2901–2912.

[24] Avinash Lakshman and Prashant Malik. 2010. Cassandra: a decentralized structured storage system. *ACM SIGOPS Operating Systems Review* 44, 2 (2010), 35–40.

[25] Steven M LaValle. 1998. Rapidly-exploring random trees: A new tool for path planning. (1998).

[26] Nancy A. Lynch. 1996. *Distributed Algorithms.* Morgan Kaufmann Publishers Inc.

[27] Pieter J Mosterman, David Escobar Sanabria, Enes Bilgin, Kun Zhang, and Justyna Zander. 2014. A heterogeneous fleet of vehicles for automated humanitarian missions. *Computing in Science & Engineering* 16, 3 (2014), 90.

[28] Adithyavairavan Murali, Tao Chen, Kalyan Vasudev Alwala, Dhiraj Gandhi, Lerrel Pinto, Saurabh Gupta, and Abhinav Gupta. 2019. PyRobot: An Open-source Robotics Framework for Research and Benchmarking. *arXiv preprint arXiv:1906.08236* (2019).

[29] B. Nitzberg and V. Lo. 1991. Distributed shared memory: a survey of issues and algorithms. *Computer* 24, 8 (aug. 1991), 52 –60. https://doi.org/10.1109/2.84877

[30] Arne Nordmann, Nico Hochgeschwender, and Sebastian Wrede. 2014. *A Survey on Domain-Specific Languages in Robotics.* Springer International Publishing, Cham, 195–206. https://doi.org/10.1007/978-3-319-11900-7_17

[31] D. Pickem, P. Glotfelter, L. Wang, M. Mote, A. Ames, E. Feron, and M. Egerstedt. 2017. The Robotarium: A remotely accessible swarm robotics research testbed. In *2017 IEEE International Conference on Robotics and Automation (ICRA).* 1699–1706. https://doi.org/10.1109/ICRA.2017.7989200

[32] Carlo Pinciroli, Adam Lee-Brown, and Giovanni Beltrame. 2015. Buzz: An Extensible Programming Language for Self-Organizing Heterogeneous Robot Swarms. *CoRR* abs/1507.05946 (2015). http://arxiv.org/abs/1507.05946

[33] J Norberto Pires and JMG Sá Da Costa. 2000. Object-oriented and distributed approach for programming robotic manufacturing cells. *Robotics and computer-integrated manufacturing* 16, 1 (2000), 29–42.

[34] Andr Platzer. 2018. *Logical Foundations of Cyber-Physical Systems* (1st ed.). Springer Publishing Company, Incorporated.

[35] Morgan Quigley, Ken Conley, Brian P. Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Y. Ng. 2009. ROS: an open-source Robot Operating System. In *ICRA Workshop on Open Source Software*.

[36] Redmond R Shamshiri, Cornelia Weltzien, Ibrahim A Hameed, Ian J Yule, Tony E Grift, Siva K Balasundram, Lenka Pitonakova, Desa Ahmad, and Girish Chowdhary. 2018. Research and development in agricultural robotics: A perspective of digital farming. (2018).

[37] Grigore Rosu and Traian Florin Serbanuta. 2014. K Overview and SIMPLE Case Study. In *Proceedings of International K Workshop (K'11) (ENTCS)*, Vol. 304. Elsevier, 3–56. https://doi.org/10.1016/j.entcs.2014.05.002

[38] David St-Onge, Vivek Shankar Varadharajan, Guannan Li, Ivan Svogor, and Giovanni Beltrame. 2017. ROS and Buzz: consensus-based behaviors for heterogeneous teams. *CoRR* abs/1710.08843 (2017). arXiv:1710.08843 http://arxiv.org/abs/1710.08843

[39] Sebastian Thrun et al. 2002. Robotic mapping: A survey. *Exploring artificial intelligence in the new millennium* 1, 1-35 (2002), 1.

[40] Brian C Williams, Michel D Ingham, Seung H Chung, and Paul H Elliott. 2003. Model-based programming of intelligent embedded systems and robotic space explorers. *Proc. IEEE* 91, 1 (2003), 212–237.

[41] Damien Zufferey. 2017. The REACT language for robotics.